

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M Vishnu Vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT3-COMPARATIVE ANALYSIS

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Prim's vs Kruskal's Algorithm
(MST Construction)
100 marks

1/1 answered

Q1 Prim's vs Kruskal's Algorithm (MST Construction)

100 marks

Given a graph, construct MST using Prim's and Kruskal's methods. Compare edge selection strategies. Analyze efficiency in dense vs sparse graphs. Generate insights on MST algorithm suitability.

Your Code

```
1 import java.util.*;
2
3 public class Main {
4
5     static int V;
6
7     // ----- Kruskal's Algorithm -----
8     static class Edge implements Comparable<Edge> {
9         int src, dest, weight;
10        Edge(int s, int d, int w) { src = s; dest = d; weight = w; }
11        public int compareTo(Edge o) { return this.weight - o.weight; }
12    }
13
14    static int find(int[] parent, int i) {
15        if (parent[i] != i) parent[i] = find(parent, parent[i]);
16        return parent[i];
17    }
18
19    static void union(int[] parent, int[] rank, int x, int y) {
20        int rx = find(parent, x), ry = find(parent, y);
21        if (rank[rx] < rank[ry]) parent[rx] = ry;
22        else if (rank[rx] > rank[ry]) parent[ry] = rx;
23        else { parent[ry] = rx; rank[rx]++; }
24    }
25
26    static int kruskalMST(List<Edge> edges, StringBuilder sb) {
27        Collections.sort(edges);
28        int[] parent = new int[V];
29        int[] rank = new int[V];
30        for (int i = 0; i < V; i++) parent[i] = i;
31    }
```

Input

```
5 7
0 1 2
0 3 6
1 2 3
```

Output

```
Kruskal's MST Edges:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Kruskal's edge checks: 5
```

Both build a Minimum Spanning Tree, but with different strategies:

Kruskal's Algorithm:

Sorts all edges by weight.

Uses Union-Find (Disjoint Set) to add the smallest edge that doesn't form a cycle.

Good for sparse graphs — dominated by sorting, $O(E \log E)$.

Prim's Algorithm:

Starts from one vertex, grows the MST by always adding the cheapest edge connecting the tree to a new vertex.

Uses a `key[]` array to track the minimum edge weight to reach each unvisited vertex.

With an adjacency matrix (as coded), it's $O(V^2)$ — good for dense graphs.

In the code:

`kruskalMST()` sorts edges, then greedily unions disjoint components, skipping edges that would form a cycle.

`primMST()` repeatedly picks the unvisited vertex with the smallest `key[]` value, adds its

edge, then updates `key[]` for its neighbors.

Both track a comparison/edge-check counter so you can discuss efficiency.

The final section computes graph density ($2E / V(V-1)$) and recommends Kruskal's for sparse graphs, Prim's for dense ones — this is the "comparative analysis" the task asks for